

OBJECT ORIENTED PROGRAMMING WITH C++

Unit-III: Objects, Classes, and Arrays

Comprehensive Study Notes

UNIT-III: OBJECTS, CLASSES, AND ARRAYS

1. Classes and Objects

1.1 Introduction to Classes and Objects

A **class** is a user-defined data type that acts as a blueprint for creating objects. An **object** is an instance of a class.

1.2 Defining a Class

Syntax:

```
class class_name {  
    // data members  
    // member functions  
};
```

Example: Simple Class

```
#include <iostream>  
#include <string>  
using namespace std;  
  
class Student {  
    int rollNo;  
    string name;  
    float marks;  
};  
  
int main() {  
    Student s1;  
    return 0;  
}
```

Output:

Roll No: 101
Name: Rahul
Marks: 85.5

1.3 Access Specifiers

Specifier	Access Level
private	Accessible only within the class (default)
public	Accessible from anywhere
protected	Accessible within the class and derived classes

1.4 Member Functions (Methods)

Member functions can be defined:

1. **Inside the class** (implicitly inline)
2. **Outside the class** using the scope resolution operator

Example: Function Defined Outside Class

```
#include <iostream>
#include <string>
using namespace std;

class Student {
    int rollNo;
    string name;
    float marks;
public:
    void setData(int r, string n, float m);
    void display();
};

// Function definitions outside class
void Student::setData(int r, string n, float m) {
    rollNo = r;
    name = n;
    marks = m;
}
```

```
}

void Student::display() {
    cout << "Roll No: " << rollNo << endl;
    cout << "Name: " << name << endl;
    cout << "Marks: " << marks << endl;
}

int main() {
    Student s1;
    s1.setData(101, "Rahul", 85.5);
    s1.display();
    return 0;
}
```

1.5 C++ Objects as Physical Objects

Objects can represent real-world physical entities.

```
class Car {
    string brand;
    string model;
    int year;
public:
    void setDetails(string b, string m, int y) {
        brand = b;
        model = m;
        year = y;
    }
    void display() {
        cout << brand << " " << model << " (" << year << ")" << endl;
    }
};

int main() {
    Car c1;
    c1.setDetails("Toyota", "Camry", 2022);
    c1.display();
    return 0;
}
```

1.6 C++ Objects as Data Types

Objects can be used as data types in arrays, structures, and as function parameters.

```
class Point {  
    int x, y;  
public:  
    void setPoint(int a, int b) { x = a; y = b; }  
    void display() { cout << "(" << x << ", " << y << ")" << endl; }  
};  
  
int main() {  
    Point p1, p2, p3;  
    p1.setPoint(3, 4);  
    p2.setPoint(5, 6);  
    p3.setPoint(8, 10);  
    p1.display();  
    p2.display();  
    p3.display();  
    return 0;  
}
```

Output:

```
p1: (3, 4)  
p2: (5, 6)  
p3: (8, 10)
```

2. Constructors and Destructors

2.1 What is a Constructor?

A **constructor** is a special member function that is automatically called when an object is created. It has the same name as the class and no return type.

Characteristics:

- Same name as class
- No return type (not even void)
- Automatically invoked when object is created
- Can be overloaded
- Can have default arguments

2.2 Types of Constructors

Type	Description
Default Constructor	No parameters
Parameterized Constructor	Accepts parameters for initialization
Copy Constructor	Initializes using another object
Overloaded Constructor	Multiple constructors with different parameters

2.3 Default Constructor

A constructor with no parameters.

```
class Student {  
    int rollNo;  
    string name;  
public:  
    Student() { // Default constructor  
        rollNo = 0;  
        name = "Unknown";  
    }  
};
```

```
        cout << "Default constructor called" << endl;
    }
};

int main() {
    Student s1;
    return 0;
}
```

Output:

```
Default constructor called
Roll No: 0, Name: Unknown
```

2.4 Parameterized Constructor

A constructor with parameters to initialize object with specific values.

```
class Student {
    int rollNo;
    string name;
public:
    Student(int r, string n) { // Parameterized constructor
        rollNo = r;
        name = n;
        cout << "Parameterized constructor called" << endl;
    }
};

int main() {
    Student s1(101, "Rahul");
    Student s2(102, "Priya");
    return 0;
}
```

Output:

```
Parameterized constructor called
Roll No: 101, Name: Rahul
Parameterized constructor called
Roll No: 102, Name: Priya
```

2.5 Constructor with Default Arguments

```
class Student {
    int rollNo;
    string name;
public:
    Student(int r = 0, string n = "Unknown") {
        rollNo = r;
        name = n;
    }
    void display() {
        cout << "Roll No: " << rollNo << ", Name: " << name << endl;
    }
};

int main() {
    Student s1;
    Student s2(101);
    Student s3(102, "Rahul");
    s1.display();
    s2.display();
    s3.display();
    return 0;
}
```

Output:

```
Roll No: 0, Name: Unknown
Roll No: 101, Name: Unknown
Roll No: 102, Name: Rahul
```

2.6 Overloaded Constructors

A class can have multiple constructors with different parameters.

```
class Rectangle {
    int length, breadth;
public:
    Rectangle() { length = 0; breadth = 0; }
    Rectangle(int s) { length = s; breadth = s; }
    Rectangle(int l, int b) { length = l; breadth = b; }
    int area() { return length * breadth; }
```



```
};

int main() {
    Rectangle r1;
    Rectangle r2(5);
    Rectangle r3(4, 6);
    cout << "Area of r1: " << r1.area() << endl;
    cout << "Area of r2: " << r2.area() << endl;
    cout << "Area of r3: " << r3.area() << endl;
    return 0;
}
```

Output:

```
Area of r1: 0
Area of r2: 25
Area of r3: 24
```

2.7 Copy Constructor

A copy constructor initializes an object using another object of the same class.

```
class Student {
    int rollNo;
    string name;
public:
    Student(int r, string n) {
        rollNo = r;
        name = n;
    }
    Student(const Student &s) { // Copy constructor
        rollNo = s.rollNo;
        name = s.name;
        cout << "Copy constructor called" << endl;
    }
    void display() {
        cout << "Roll No: " << rollNo << ", Name: " << name << endl;
    }
};

int main() {
    Student s1(101, "Rahul");
    Student s2 = s1; // Copy constructor
```

```
Student s3(s1); // Copy constructor
s1.display();
s2.display();
s3.display();
return 0;
}
```

Output:

```
Copy constructor called
Copy constructor called
Roll No: 101, Name: Rahul
Roll No: 101, Name: Rahul
Roll No: 101, Name: Rahul
```

2.8 Destructor

A **destructor** is a special member function that is automatically called when an object goes out of scope. It has the same name as the class with a tilde (~) prefix.

```
class MyClass {
public:
    MyClass() { cout << "Constructor called" << endl; }
    ~MyClass() { cout << "Destructor called" << endl; }
};

int main() {
    MyClass obj;
    cout << "Inside main" << endl;
    return 0;
}
```

Output:

```
Constructor called
Inside main
Destructor called
```

3. Passing Objects to Functions

3.1 Passing Objects to Functions

Objects can be passed to functions by value, by reference, or by pointer.

```
class Student {
    string name;
    float marks;
public:
    void setData(string n, float m) { name = n; marks = m; }
    float getMarks() { return marks; }
    string getName() { return name; }
};

void compareMarks(Student s1, Student s2) {
    if (s1.getMarks() > s2.getMarks())
        cout << s1.getName() << " has higher marks" << endl;
    else
        cout << s2.getName() << " has higher marks" << endl;
}

int main() {
    Student s1, s2;
    s1.setData("Rahul", 85);
    s2.setData("Priya", 92);
    compareMarks(s1, s2);
    return 0;
}
```

Output:

```
Priya has higher marks
```

4. Returning Objects from Functions

4.1 Returning Objects

Functions can return objects of a class.

```
class Point {  
    int x, y;  
public:  
    void setPoint(int a, int b) { x = a; y = b; }  
    void display() { cout << "(" << x << ", " << y << ")" << endl; }  
    Point add(Point p) {  
        Point temp;  
        temp.x = x + p.x;  
        temp.y = y + p.y;  
        return temp;  
    }  
};  
  
int main() {  
    Point p1, p2, p3;  
    p1.setPoint(3, 4);  
    p2.setPoint(5, 6);  
    p3 = p1.add(p2);  
    p1.display();  
    p2.display();  
    p3.display();  
    return 0;  
}
```

Output:

```
p1: (3, 4)  
p2: (5, 6)  
p3: (8, 10)
```

5. Arrays in C++

5.1 Array Fundamentals

An **array** is a collection of similar data elements stored in contiguous memory locations.

Defining an Array:

```
data_type array_name[size];

// Examples
int marks[5];
float prices[10];
char name[20];
```

5.2 Accessing Array Elements

```
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++) {
        cout << "arr[" << i << "] = " << arr[i] << endl;
    }
    return 0;
}
```

5.3 Initializing Arrays

Method	Example	Description
1. At declaration	<code>int arr1[5] = {10, 20, 30, 40, 50};</code>	Full initialization
2. Partial	<code>int arr2[5] = {10, 20};</code>	Rest initialized to 0
3. Implicit size	<code>int arr3[] = {10, 20, 30};</code>	Size = 5

4. All zeros	<code>int arr4[5] = {0};</code>	All elements 0
5. Using loop	<code>for (int i = 0; i < 5; i++) cin >> arr5[i];</code>	Runtime input

5.4 Multidimensional Arrays

```
int main() {
    int matrix[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };
    cout << "Element at (1,2): " << matrix[1][2] << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++)
            cout << matrix[i][j] << " ";
        cout << endl;
    }
    return 0;
}
```

Output:

```
Element at (1,2): 7
1 2 3 4
5 6 7 8
9 10 11 12
```

5.5 Passing Arrays to Functions

```
// Function to find sum of array elements
int sumArray(int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++)
        sum += arr[i];
    return sum;
}

// Function to modify array
void doubleArray(int arr[], int size) {
```

```
        for (int i = 0; i < size; i++)
            arr[i] *= 2;
    }

    // Function for 2D array
    void displayMatrix(int matrix[][3], int rows) {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < 3; j++)
                cout << matrix[i][j] << " ";
            cout << endl;
        }
    }

    int main() {
        int arr[5] = {10, 20, 30, 40, 50};
        cout << "Sum: " << sumArray(arr, 5) << endl;
        doubleArray(arr, 5);
        cout << "After doubling: ";
        for (int i = 0; i < 5; i++)
            cout << arr[i] << " ";
        cout << endl;

        int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};
        displayMatrix(matrix, 2);
        return 0;
    }
```

Output:

```
Sum: 150
After doubling: 20 40 60 80 100
1 2 3
4 5 6
```

5.6 Array of Objects

```
#include <iostream>
#include <string>
using namespace std;

class Student {
    int rollNo;
    string name;
```

```
        float marks;
public:
    void setData(int r, string n, float m) {
        rollNo = r;
        name = n;
        marks = m;
    }
    void display() {
        cout << "Roll No: " << rollNo << ", Name: " << name
            << ", Marks: " << marks << endl;
    }
};

int main() {
    Student s[3];
    s[0].setData(101, "Rahul", 85.5);
    s[1].setData(102, "Priya", 92.0);
    s[2].setData(103, "Amit", 78.5);
    for (int i = 0; i < 3; i++)
        s[i].display();
    return 0;
}
```


6. Strings in C++

6.1 String Variables

In C++, strings can be handled using:

1. **C-style strings** (character arrays)
2. **C++ string class** (std::string)

C-Style Strings:

```
#include <cstring>

int main() {
    char str1[20] = "Hello";
    char str2[20] = "World";
    cout << "Length of str1: " << strlen(str1) << endl;
    strcat(str1, str2);
    cout << "Concatenated: " << str1 << endl;
    return 0;
}
```

6.2 C++ String Class

The `std::string` class provides a safer and more convenient way to handle strings.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s1 = "Hello";
    string s2 = "World";
    string s3 = s1 + " " + s2;
    cout << "s3: " << s3 << endl;
    cout << "Length: " << s3.length() << endl;
    return 0;
}
```

6.3 String Class Functions

Function	Description	Example
length()	Returns string length	s.length()
size()	Returns string size	s.size()
empty()	Checks if empty	s.empty()
clear()	Clears the string	s.clear()
append(str)	Appends string	s.append("Hello")
insert(pos, str)	Inserts at position	s.insert(3, "X")
erase(pos, len)	Erases characters	s.erase(2, 3)
replace(pos, len, str)	Replaces substring	s.replace(2, 3, "XYZ")
find(str)	Finds substring	s.find("abc")
substr(pos, len)	Extracts substring	s.substr(2, 5)
c_str()	Converts to C-string	s.c_str()

6.4 Avoiding Buffer Overflow

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name;
    cout << "Enter full name: ";
    getline(cin, name); // Reads entire line safely
    cout << "Name: " << name << endl;
    return 0;
}
```

6.5 String Constants

```
int main() {  
    const string GREETING = "Hello, World!";  
    string str = GREETING;  
    cout << str << endl;  
    return 0;  
}
```

6.6 Array of Strings

```
int main() {  
    string names[] = {"Rahul", "Priya", "Amit", "Sneha"};  
    for (int i = 0; i < 4; i++)  
        cout << names[i] << endl;  
    return 0;  
}
```

6.7 String as Class Members

```
class Person {  
    string name;  
    int age;  
public:  
    void setData(string n, int a) { name = n; age = a; }  
    void display() { cout << "Name: " << name << ", Age: " << age << endl; }  
};  
  
int main() {  
    Person p;  
    p.setData("Rahul", 20);  
    p.display();  
    return 0;  
}
```

6.8 Standard C++ String Class - Complete Example

```
#include <iostream>  
#include <string>  
#include <algorithm>  
using namespace std;
```

```
int main() {  
    string str = "Hello, World!";  
    transform(str.begin(), str.end(), str.begin(), ::toupper);  
    cout << "Uppercase: " << str << endl;  
    transform(str.begin(), str.end(), str.begin(), ::tolower);  
    cout << "Lowercase: " << str << endl;  
    size_t pos = str.find("world");  
    if (pos != string::npos)  
        cout << "Found at position: " << pos << endl;  
    return 0;  
}
```

Questions

Short Answer Questions (2-5 Marks)

1. What is a class? What is an object?
2. What are access specifiers in C++?
3. What is a constructor? What are its types?
4. What is a copy constructor?
5. What is a destructor?
6. What is an array? How is it declared?
7. What is the difference between C-style string and C++ string class?
8. How do you pass arrays to functions?
9. What is an array of objects?
10. What is the `getline()` function?

Long Answer Questions (10 Marks)

1. Explain classes and objects with suitable examples.
2. Explain constructors and their types with examples.
3. Explain arrays in C++ with different operations.
4. Explain strings in C++ with various functions.
5. Explain passing and returning objects to/from functions.

Program Writing Questions (5 Marks)

1. Write a class to represent a Student with necessary data members and member functions.
2. Write a program to accept and display details of 5 students using array of objects.
3. Write a program to demonstrate constructors in C++.
4. Write a program to concatenate and compare strings.

Objective Questions (1 Mark)

1. A constructor has: (a) Return type (b) No return type **(b)** (c) Both (d) None
2. The default access specifier in C++ is: (a) public (b) private **(b)** (c) protected (d) None
3. The `getline()` function is used to: (a) Read one character (b) Read a line **(b)** (c) Read a number (d) None
4. The length of string can be found using: (a) `length()` (b) `size()` (c) Both **(c)** (d) None
5. Copy constructor is called when: (a) Object created (b) Object copied **(b)** (c) Both (d) None

QUICK REVISION

Class Structure

```
class ClassName {  
    // data members  
    // member functions  
};
```

Constructor Types

Type	Purpose
Default Constructor	No parameters, default initialization
Parameterized Constructor	Accepts values for initialization
Copy Constructor	Initializes using another object
Overloaded Constructor	Multiple constructors in same class

String Functions

Function	Purpose
length()	Length of string
append()	Appends string
insert()	Inserts at position
erase()	Erases characters
replace()	Replaces substring

find()	Finds substring
substr()	Extracts substring

End of Unit-III Notes